

EMBEDDED SYSTEMS & IoT DEVICE DESIGN TASK-1

REPORT

Submitted By: Enagandla Thrisha
BVRIT - EEE Department

1. INTRODUCTION TO EMBEDDED SYSTEMS

An embedded system is a specialized computing system designed to perform a dedicated function within a larger device. It consists of hardware and software components that work together to perform specific tasks efficiently and reliably. Embedded systems are widely used in consumer electronics, automotive systems, healthcare devices, industrial automation, and IoT applications.



Real-World Applications

Smart Homes: Smart lighting, security systems.

Automotive Systems: ABS, airbags, engine control units.

Healthcare Devices: Patient monitors, blood pressure monitors.

Robotics: Industrial robots and autonomous systems.

Microcontroller vs Microprocessor

A **microcontroller** is an entire computer on a single chip, containing a CPU, memory, and input/output ports. A **microprocessor** is just the central processing unit (CPU) and it relies on external chips for memory and peripherals.

- **Microcontrollers** are used for **dedicated tasks** (like in microwaves or remote controls). They are low-power, compact, and ready to use.
- **Microprocessors** are used for **general computing** (like in laptops or servers). They are much faster and more powerful but consume more power.

2. COMPONENT STUDY

Role of microcontrollers

Arduino, ESP32, and Raspberry Pi Pico are all popular embedded electronics platforms, but they serve completely different roles based on processing power, connectivity, and hardware control capabilities.

- **Arduino: The Beginner & Prototyping Standard**

The primary role of the traditional Arduino platform (like the classic Arduino Uno) is to act as an entry-level platform for learning electronics and building simple, reliable hardware controllers.

- **ESP32: The Connected IoT Powerhouse**

The primary role of the ESP32 is wireless communication and internet-connected automation. It bridges the gap between simple microcontrollers and full computers.

- **Raspberry Pi Pico: The Low-Cost, Precise Performance Chip**

The primary role of the Raspberry Pi Pico (built on the custom RP2040 or newer RP2350 chips) is to deliver cost-effective computing power alongside specialized hardware flexibility. Unlike a standard Raspberry Pi, the Pico is a bare-metal microcontroller that does not run Linux.



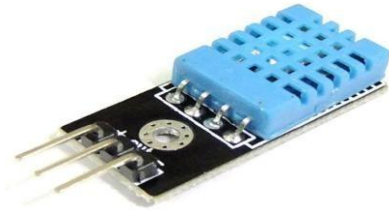
BASIC SENSORS

Temperature, motion, and light sensors are essential hardware components that act as the "eyes" and "feelers" of smart devices. They monitor environmental changes and translate physical conditions into electronic signals, enabling automation, energy savings, and security.

Temperature Sensors

A temperature sensor measures heat or cold, converting it into readable electrical signals.

Common types include thermistors (which change resistance with temperature), thermocouples (which generate voltage), and RTDs (which measure resistance precisely)



Motion Sensors

Motion sensors are electronic devices designed to detect physical movement within a designated area and trigger automated responses like sounding alarms, turning on lights, or sending smartphone notifications. They serve as the foundation for modern home security, energy conservation, and automated smart living.



Light Sensors

A light sensor (or photodetector) is a device that detects the presence or intensity of light and converts it into an electrical signal. They power everyday automation, from turning on streetlights at dusk to auto-adjusting your smartphone's display.



3. Mini Project – Smart Temperature Monitoring System

Objective

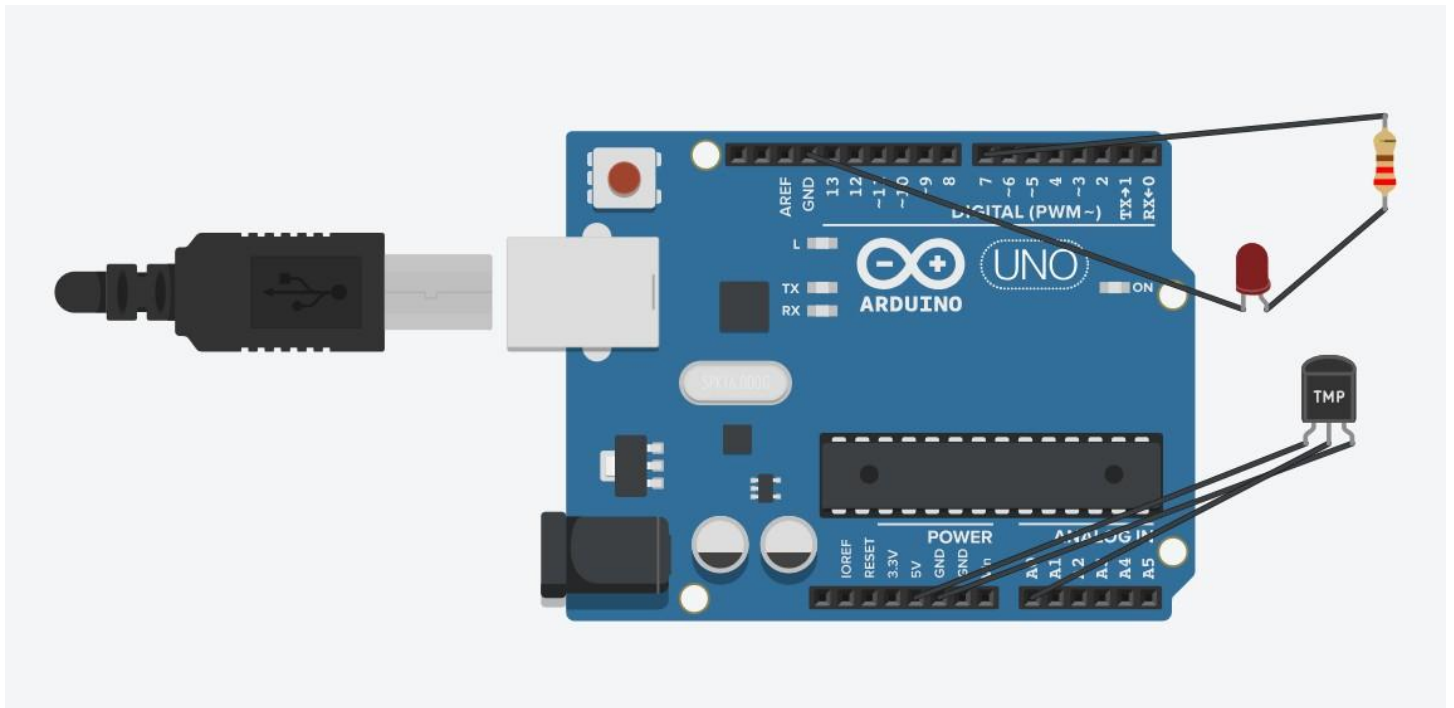
The objective of this project is to monitor environmental temperature using a temperature sensor and provide an alert whenever the temperature exceeds a predefined threshold value.

Components Required

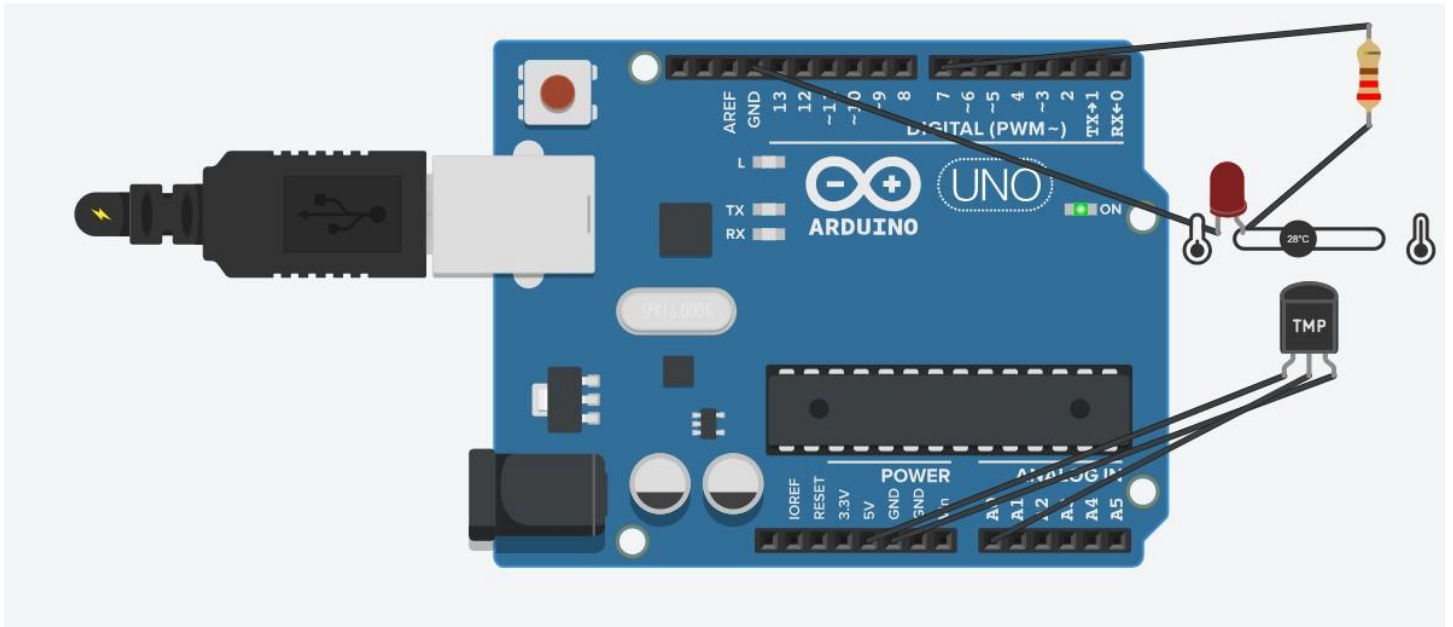
- Arduino Uno
- LM35/TMP36 Temperature Sensor
- LED
- 220Ω Resistor
- Breadboard
- Jumper Wires

Circuit Diagram

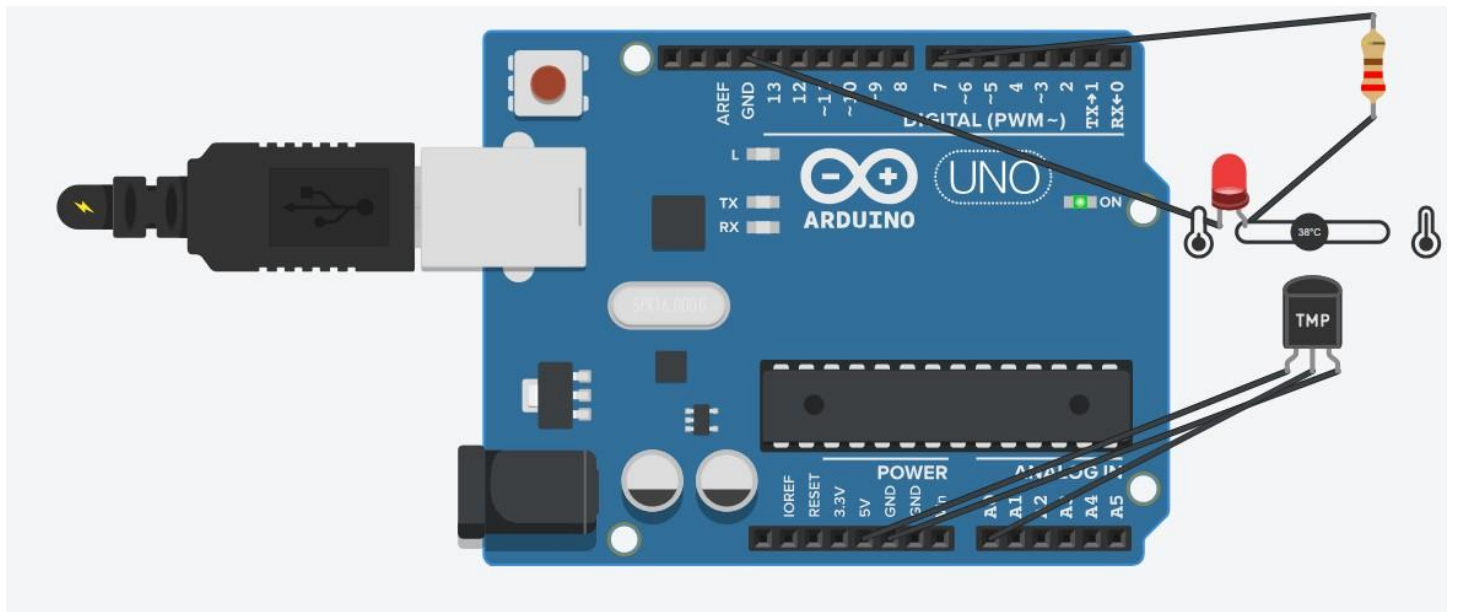
The Smart Temperature Monitoring System was designed and simulated using Tinkercad Circuits. The simulation verified the functionality of the system by monitoring temperature values and controlling the LED based on the predefined threshold temperature.



Circuit Diagram of Smart Temperature Monitoring System



Simulation Output at 28°C (LED OFF)



Simulation Output at 38°C (LED ON)

Working Principle

The Smart Temperature Monitoring System continuously measures the surrounding temperature using a temperature sensor. The sensor converts temperature into an electrical signal and sends it to the Arduino Uno.

The Arduino reads the sensor data and compares the measured temperature with a predefined threshold value of 30°C. If the temperature exceeds the threshold, the Arduino turns ON the LED to indicate a high-temperature condition. When the temperature falls below the threshold value, the LED turns OFF. This system provides a simple and effective method for monitoring temperature and generating alerts when abnormal conditions are detected.

Arduino Program

```
// Smart Temperature Monitoring System const int tempPin =
A0; // Temperature sensor connected to A0 const int ledPin =
7; // LED connected to pin 7

void setup() { pinMode(ledPin, OUTPUT); // Set LED
pin as output
  Serial.begin(9600); // Start serial communication
}

void loop() { int sensorValue = analogRead(tempPin); // Read
sensor value float voltage = sensorValue * (5.0 / 1023.0); //
Convert to voltage float temperature = (voltage - 0.5) * 100.0; //
Convert to Celsius

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" C");

  if (temperature > 30) { // Check threshold
digitalWrite(ledPin, HIGH); // Turn ON LED
  } else { digitalWrite(ledPin, LOW); // Turn
OFF LED
  }

  delay(1000); // Wait 1 second
}
```

Code Explanation:

- A0 is used to read temperature data from the TMP36 sensor.
- Pin 7 controls the LED.
- The Arduino converts the sensor reading into temperature values.
- Temperature readings are displayed on the Serial Monitor.
- If the temperature exceeds 30°C, the LED turns ON.
- If the temperature is 30°C or below, the LED remains OFF.
- The process repeats every 1 second.

Output

- LED remains OFF when temperature is below 30°C.
- LED turns ON when temperature exceeds 30°C.
- Temperature values can be monitored through the Serial Monitor.

Applications

- Weather Monitoring Stations
- Industrial Temperature Monitoring
- Smart Agriculture Systems
- Home Automation Systems
- Server Room Temperature Monitoring

Advantages

- Simple and low-cost design
- Easy to implement and maintain
- Real-time temperature monitoring
- Can be expanded for IoT-based remote monitoring

Conclusion

The Smart Temperature Monitoring System successfully demonstrates the use of embedded systems and sensors for environmental monitoring. The project highlights how microcontrollers can process sensor data and provide automated alerts, making it useful for various real-world applications.